

GROUP ASSIGNMENT

URBAN COMPLAINT MANAGEMENT

Problem Statement: Design, implement, and evaluate a menu-driven, object-oriented C++ application for an Urban Complaint Management and Service Request System to automate the registration, categorization, prioritization, assignment, status tracking, and resolution of civic complaints such as road damage, water supply, waste management, drainage, and streetlight issues. The system shall model citizens, complaints, departments, and service requests as encapsulated classes and manage multiple objects using suitable data types, control structures, arrays, and static members. Different complaint types shall be organized using inheritance and virtual functions to support runtime polymorphism and customized complaint processing. The application shall demonstrate default, parameterized, and copy constructors, destructors, pointers, and at least two meaningful operator overloads for comparing complaints and generating reports. A menu-driven interface shall provide complaint registration, department assignment, status updates, complaint comparison, resolution tracking, and service-performance report generation.

Aim:

To design and implement a menu-driven Urban Complaint Management and Service Request System using C++ that automates the registration, categorization, prioritization, department assignment, status tracking, resolution monitoring, complaint comparison, and service-performance reporting of urban civic complaints using object-oriented programming concepts such as encapsulation, constructors, destructors, inheritance, virtual functions, polymorphism, pointers, static members, arrays, and operator overloading.

Pseudo code:

START

Create an array of Complaint pointers.

Initialize complaint count to 0.

Display the main menu repeatedly.

WHILE user does not select Exit:

 Display menu options.

 Read user's choice.

 IF choice = 1:

 Read complaint ID.

 Read citizen name.

 Read complaint description.

 Read location.

 Display complaint types.

 Read complaint type.

 IF type is Road:

 Create RoadComplaint object.

 ELSE IF type is Water:

 Create WaterComplaint object.

 ELSE IF type is Waste:

 Create WasteComplaint object.

 ELSE IF type is Drainage:

Create DrainageComplaint object.

ELSE IF type is Streetlight:

Create StreetlightComplaint object.

Store object in complaint array.

Determine priority using virtual function.

Process complaint using polymorphism.

ELSE IF choice = 2:

Display all registered complaints.

ELSE IF choice = 3:

Read complaint ID.

Find the complaint.

Assign the appropriate department.

Change status to Assigned.

ELSE IF choice = 4:

Read complaint ID.

Display status options.

Update complaint status.

ELSE IF choice = 5:

Read complaint ID.

Display complaint details.

Check whether complaint is resolved or pending.

ELSE IF choice = 6:

Read two complaint IDs.

Compare complaints using overloaded operators.

Display the complaint with higher priority.

ELSE IF choice = 7:

Count total complaints.

Count complaints by category.

Count resolved and pending complaints.

Calculate resolution rate.

Display performance report.

ELSE IF choice = 8:

Display total complaints using static member.

ELSE IF choice = 9:

Delete dynamically allocated complaint objects.

Display exit message.

EXIT LOOP.

ELSE:

Display invalid choice.

END WHILE

STOP

Code:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// ===== CITIZEN CLASS =====
```

```
class Citizen
```

```
{
```

```
private:
```

```
    int citizenId;
```

```
    string name;
```

```
    string phone;
```

```
public:
```

```
Citizen()
```

```
{  
    citizenId = 0;  
    name = "";  
    phone = "";  
}
```

```
Citizen(int id, string n, string p)
```

```
{  
    citizenId = id;  
    name = n;  
    phone = p;  
}
```

```
void displayCitizen()
```

```
{  
    cout << "\nCitizen ID: " << citizenId;  
    cout << "\nName: " << name;  
    cout << "\nPhone: " << phone << endl;  
}
```

```
int getCitizenId()
```

```
{
```

```

        return citizenId;
    }

    string getName()
    {
        return name;
    }
};

// ===== ABSTRACT BASE CLASS =====

class Complaint
{
protected:
    int complaintId;
    string citizenName;
    string description;
    string location;
    string department;
    string status;

    static int totalComplaints;

```

public:

// Default Constructor

Complaint()

```
{  
    complaintId = 0;  
    citizenName = "";  
    description = "";  
    location = "";  
    department = "Not Assigned";  
    status = "Registered";  
}
```

// Parameterized Constructor

Complaint(int id, string name, string desc, string loc)

```
{  
    complaintId = id;  
    citizenName = name;  
    description = desc;  
    location = loc;  
    department = "Not Assigned";  
    status = "Registered";  
}
```



```
        totalComplaints++;  
    }
```

```
// Copy Constructor
```

```
Complaint(const Complaint &c)
```

```
{  
    complaintId = c.complaintId;  
    citizenName = c.citizenName;  
    description = c.description;  
    location = c.location;  
    department = c.department;  
    status = c.status;
```

```
        totalComplaints++;  
    }
```

```
// Virtual Destructor
```

```
virtual ~Complaint()
```

```
{  
  
}
```

```
int getComplaintId() const
```

```
{
```

```
        return complaintId;
    }
```

```
string getStatus() const
{
    return status;
}
```

```
string getDepartment() const
{
    return department;
}
```

```
void assignDepartment()
{
    department = getDepartmentName();
    status = "Assigned";
}
```

```
void updateStatus(string newStatus)
{
    status = newStatus;
}
```

```
static int getTotalComplaints()
```

```
{
```

```
    return totalComplaints;
```

```
}
```

```
// Pure Virtual Functions
```

```
virtual string getType() const = 0;
```

```
virtual string getPriority() const = 0;
```

```
virtual string getDepartmentName() const = 0;
```

```
virtual void processComplaint() const = 0;
```

```
// Operator Overloading: Compare Complaint ID
```

```
bool operator==(const Complaint &other) const
```

```
{
```

```
    return complaintId == other.complaintId;
```

```
}
```

```
// Operator Overloading: Compare Priority
```

```
bool operator>(const Complaint &other) const
```

```
{
```

```
    int p1 = getPriorityValue();
```

```
    int p2 = other.getPriorityValue();
```

```
    return p1 > p2;
}
```

```
int getPriorityValue() const
{
    string p = getPriority();

    if (p == "CRITICAL")
        return 4;
    else if (p == "HIGH")
        return 3;
    else if (p == "MEDIUM")
        return 2;
    else
        return 1;
}
```

```
virtual void display() const
{
    cout << "\n-----";
    cout << "\nComplaint ID : " << complaintId;
    cout << "\nCitizen Name : " << citizenName;
```

```

        cout << "\nType      : " << getType();

        cout << "\nDescription : " << description;

        cout << "\nLocation   : " << location;

        cout << "\nPriority    : " << getPriority();

        cout << "\nDepartment : " << department;

        cout << "\nStatus      : " << status;

        cout << "\n-----\n";

    }

};

int Complaint::totalComplaints = 0;

// ===== ROAD COMPLAINT =====

class RoadComplaint : public Complaint
{
public:

    RoadComplaint(int id, string name, string desc, string loc)
        : Complaint(id, name, desc, loc)

    {

    }

    string getType() const

```

```

    {
        return "Road Damage";
    }

string getPriority() const
{
    return "HIGH";
}

string getDepartmentName() const
{
    return "Road Maintenance Department";
}

void processComplaint() const
{
    cout << "\nProcessing Road Damage Complaint...";
}

};

// ===== WATER COMPLAINT =====

class WaterComplaint : public Complaint

```

```
{  
public:  
    WaterComplaint(int id, string name, string desc, string loc)  
        : Complaint(id, name, desc, loc)  
    {  
    }  
  
    string getType() const  
    {  
        return "Water Supply";  
    }  
  
    string getPriority() const  
    {  
        return "CRITICAL";  
    }  
  
    string getDepartmentName() const  
    {  
        return "Water Supply Department";  
    }  
  
    void processComplaint() const
```

```
{  
    cout << "\nProcessing Water Supply Complaint...";  
}  
};
```

```
// ===== WASTE COMPLAINT =====
```

```
class WasteComplaint : public Complaint
```

```
{
```

```
public:
```

```
    WasteComplaint(int id, string name, string desc, string loc)
```

```
        : Complaint(id, name, desc, loc)
```

```
{
```

```
}
```

```
    string getType() const
```

```
{
```

```
        return "Waste Management";
```

```
}
```

```
    string getPriority() const
```

```
{
```

```
        return "MEDIUM";
```



```
}
```

```
string getDepartmentName() const
```

```
{
```

```
    return "Sanitation Department";
```

```
}
```

```
void processComplaint() const
```

```
{
```

```
    cout << "\nProcessing Waste Management Complaint...";
```

```
}
```

```
};
```

```
// ===== DRAINAGE COMPLAINT =====
```

```
class DrainageComplaint : public Complaint
```

```
{
```

```
public:
```

```
    DrainageComplaint(int id, string name, string desc, string loc)
```

```
        : Complaint(id, name, desc, loc)
```

```
{
```

```
}
```

```

string getType() const
{
    return "Drainage Issue";
}

string getPriority() const
{
    return "HIGH";
}

string getDepartmentName() const
{
    return "Drainage Department";
}

void processComplaint() const
{
    cout << "\nProcessing Drainage Complaint...";
}

};

// ===== STREETLIGHT COMPLAINT =====

```

```
class StreetlightComplaint : public Complaint
{
public:
    StreetlightComplaint(int id, string name, string desc, string loc)
        : Complaint(id, name, desc, loc)
    {
    }

    string getType() const
    {
        return "Streetlight Issue";
    }

    string getPriority() const
    {
        return "MEDIUM";
    }

    string getDepartmentName() const
    {
        return "Electrical Department";
    }
}
```

```

void processComplaint() const
{
    cout << "\nProcessing Streetlight Complaint...";
}

};

// ===== MAIN PROGRAM =====

int main()
{
    const int MAX = 100;

    Complaint *complaints[MAX];

    int complaintCount = 0;

    int choice;

    do
    {
        cout << "\n\n===== ";

        cout << "\n  URBAN COMPLAINT MANAGEMENT SYSTEM";

        cout << "\n===== ";
    }

```

```
cout << "\n1. Register Complaint";

cout << "\n2. Display All Complaints";

cout << "\n3. Assign Department";

cout << "\n4. Update Complaint Status";

cout << "\n5. Track Resolution";

cout << "\n6. Compare Two Complaints";

cout << "\n7. Generate Service Performance Report";

cout << "\n8. Display Statistics";

cout << "\n9. Exit";


cout << "\n\nEnter your choice: ";

cin >> choice;


switch (choice)

{

case 1:

{

if (complaintCount >= MAX)

{

cout << "\nMaximum complaints reached!";

break;

}

}
```

```
int id, type;

string name, description, location;


cout << "\nEnter Complaint ID: ";

cin >> id;


cout << "Enter Citizen Name: ";

cin >> ws;

getline(cin, name);


cout << "Enter Description: ";

getline(cin, description);


cout << "Enter Location: ";

getline(cin, location);


cout << "\nSelect Complaint Type:";

cout << "\n1. Road Damage";

cout << "\n2. Water Supply";

cout << "\n3. Waste Management";

cout << "\n4. Drainage Issue";

cout << "\n5. Streetlight Issue";
```

```
cout << "\nEnter type: ";
```

```
cin >> type;
```

```
switch (type)
```

```
{
```

```
case 1:
```

```
    complaints[complaintCount] =
```

```
        new RoadComplaint(id, name, description, location);
```

```
    break;
```

```
case 2:
```

```
    complaints[complaintCount] =
```

```
        new WaterComplaint(id, name, description, location);
```

```
    break;
```

```
case 3:
```

```
    complaints[complaintCount] =
```

```
        new WasteComplaint(id, name, description, location);
```

```
    break;
```

```
case 4:
```

```
    complaints[complaintCount] =
```

```
        new DrainageComplaint(id, name, description, location);
```

```
break;
```

```
case 5:
```

```
complaints[complaintCount] =
```

```
    new StreetlightComplaint(id, name, description, location);
```

```
break;
```

```
default:
```

```
    cout << "\nInvalid Complaint Type!";
```

```
    continue;
```

```
}
```

```
cout << "\nComplaint Registered Successfully!";
```

```
cout << "\nPriority: "
```

```
    << complaints[complaintCount]->getPriority();
```

```
complaints[complaintCount]->processComplaint();
```

```
complaintCount++;
```

```
break;
```

```
}
```


case 2:

```
{  
    if (complaintCount == 0)  
    {  
        cout << "\nNo complaints available!";  
    }  
    else  
    {  
        cout << "\n===== ALL COMPLAINTS =====\n";  
  
        for (int i = 0; i < complaintCount; i++)  
        {  
            complaints[i]->display();  
        }  
    }  
  
    break;  
}
```

case 3:

```
{  
    int id;
```

```
bool found = false;
```

```
cout << "\nEnter Complaint ID: ";
```

```
cin >> id;
```

```
for (int i = 0; i < complaintCount; i++)
```

```
{
```

```
    if (complaints[i]->getComplaintId() == id)
```

```
    {
```

```
        complaints[i]->assignDepartment();
```

```
        cout << "\nDepartment Assigned Successfully!";
```

```
        cout << "\nDepartment: "
```

```
            << complaints[i]->getDepartment();
```

```
        found = true;
```

```
        break;
```

```
    }
```

```
}
```

```
if (!found)
```

```
{
```

```
    cout << "\nComplaint not found!";
```

```
}
```

```
break;
```

```
}
```

```
case 4:
```

```
{
```

```
    int id;
```

```
    int statusChoice;
```

```
    bool found = false;
```

```
    cout << "\nEnter Complaint ID: ";
```

```
    cin >> id;
```

```
    for (int i = 0; i < complaintCount; i++)
```

```
    {
```

```
        if (complaints[i]->getComplaintId() == id)
```

```
        {
```

```
            cout << "\nSelect New Status:";
```

```
            cout << "\n1. Registered";
```

```
            cout << "\n2. Assigned";
```

```
            cout << "\n3. In Progress";
```

```
cout << "\n4. Resolved";

cout << "\n5. Closed";


cout << "\nEnter choice: ";

cin >> statusChoice;


if (statusChoice == 1)

    complaints[i]->updateStatus("Registered");


else if (statusChoice == 2)

    complaints[i]->updateStatus("Assigned");


else if (statusChoice == 3)

    complaints[i]->updateStatus("In Progress");


else if (statusChoice == 4)

    complaints[i]->updateStatus("Resolved");


else if (statusChoice == 5)

    complaints[i]->updateStatus("Closed");


else

{
```

```
    cout << "\nInvalid status!";
```

```
    break;
```

```
}
```

```
    cout << "\nStatus Updated Successfully!";
```

```
    found = true;
```

```
    break;
```

```
}
```

```
}
```

```
if (!found)
```

```
{
```

```
    cout << "\nComplaint not found!";
```

```
}
```

```
break;
```

```
}
```

```
case 5:
```

```
{
```

```
    int id;
```

```
bool found = false;
```

```
cout << "\nEnter Complaint ID: ";
```

```
cin >> id;
```

```
for (int i = 0; i < complaintCount; i++)
```

```
{
```

```
    if (complaints[i]->getComplaintId() == id)
```

```
    {
```

```
        cout << "\n===== RESOLUTION TRACKING =====";
```

```
        complaints[i]->display();
```

```
        if (complaints[i]->getStatus() == "Resolved" ||
```

```
            complaints[i]->getStatus() == "Closed")
```

```
        {
```

```
            cout << "\nComplaint has been resolved.";
```

```
        }
```

```
    else
```

```
    {
```

```
        cout << "\nComplaint is still under processing.";
```

```
    }
```

```
found = true;
```

```
        break;

    }

}

if (!found)

{

    cout << "\nComplaint not found!";

}

break;

}

case 6:

{

    int id1, id2;

    Complaint *c1 = NULL;

    Complaint *c2 = NULL;

    cout << "\nEnter First Complaint ID: ";

    cin >> id1;

    cout << "Enter Second Complaint ID: ";
```

```
cin >> id2;
```

```
for (int i = 0; i < complaintCount; i++)
```

```
{
```

```
    if (complaints[i]->getComplaintId() == id1)
```

```
        c1 = complaints[i];
```

```
    if (complaints[i]->getComplaintId() == id2)
```

```
        c2 = complaints[i];
```

```
}
```

```
if (c1 == NULL || c2 == NULL)
```

```
{
```

```
    cout << "\nOne or both complaints not found!";
```

```
}
```

```
else
```

```
{
```

```
    if (*c1 == *c2)
```

```
    {
```

```
        cout << "\nBoth complaints have the same ID.";
```

```
    }
```

```
else
```

```
{
```



```
    cout << "\nComplaints are different.";

    if (*c1 > *c2)
    {
        cout << "\nComplaint " << id1
            << " has higher priority.";
    }
    else if (*c2 > *c1)
    {
        cout << "\nComplaint " << id2
            << " has higher priority.";
    }
    else
    {
        cout << "\nBoth complaints have the same priority.";
    }
}

break;
}
```

case 7:

{

int resolved = 0;

int pending = 0;

int road = 0;

int water = 0;

int waste = 0;

int drainage = 0;

int streetlight = 0;

for (int i = 0; i < complaintCount; i++)

{

string type = complaints[i]->getType();

if (type == "Road Damage")

road++;

else if (type == "Water Supply")

water++;

else if (type == "Waste Management")

waste++;

```
else if (type == "Drainage Issue")

    drainage++;

else if (type == "Streetlight Issue")

    streetlight++;

if (complaints[i]->getStatus() == "Resolved" ||
    complaints[i]->getStatus() == "Closed")
{
    resolved++;
}
else
{
    pending++;
}
}
```

```
double resolutionRate = 0;
```

```
if (complaintCount > 0)
{
    resolutionRate =
```

```

        (double)resolved / complaintCount * 100;
    }

    cout << "\n\n=====";
    cout << "\n    SERVICE PERFORMANCE REPORT";
    cout << "\n=====";

    cout << "\nTotal Complaints    : "
        << complaintCount;

    cout << "\nResolved Complaints  : "
        << resolved;

    cout << "\nPending Complaints   : "
        << pending;

    cout << "\n\nRoad Complaints      : "
        << road;

    cout << "\nWater Complaints       : "
        << water;

    cout << "\nWaste Complaints        : "

```

```
<< waste;
```

```
cout << "\nDrainage Complaints  : "
```

```
<< drainage;
```

```
cout << "\nStreetlight Complaints : "
```

```
<< streetlight;
```

```
cout << "\n\nResolution Rate      : "
```

```
<< resolutionRate << "%";
```

```
cout << "\n=====
```

```
break;
```

```
}
```

```
case 8:
```

```
{
```

```
cout << "\n===== SYSTEM STATISTICS =====
```

```
cout << "\nTotal Complaints Registered: "
```

```
<< Complaint::getTotalComplaints();
```

```
        cout << "\nCurrent Complaints Stored: "
            << complaintCount;

        break;
    }

    case 9:

    {
        cout << "\nThank you for using the system!";

        for (int i = 0; i < complaintCount; i++)
        {
            delete complaints[i];
        }

        break;
    }

    default:

        cout << "\nInvalid choice! Please try again.";
    }

} while (choice != 9);
```

```
    return 0;
}
```

Output:

1&2. Register Complaint & Display Complaints:

```
===== ALL COMPLAINTS =====
-----
Complaint ID : 101
Citizen Name : ravi kumar
Type         : Road Damage
Description  : large pot hole near college
Location     : chennai main road
Priority     : HIGH
Department  : Not Assigned
Status      : Registered
-----
-----
Complaint ID : 102
Citizen Name : priya
Type         : Water Supply
Description  : no water supply for two days
Location     : anna nagar
Priority     : CRITICAL
Department  : Not Assigned
Status      : Registered
-----
```

```
-----  
Complaint ID : 103  
Citizen Name : arun  
Type : Waste Management  
Description : garbage not collected  
Location : t nagar  
Priority : MEDIUM  
Department : Not Assigned  
Status : Registered  
-----  
  
-----  
Complaint ID : 104  
Citizen Name : karthik  
Type : Drainage Issue  
Description : drainage water overflowing  
Location : velachery  
Priority : HIGH  
Department : Not Assigned  
Status : Registered  
-----
```

```
-----  
Complaint ID : 105  
Citizen Name : meena  
Type : Streetlight Issue  
Description : streetlight not working  
Location : adyar  
Priority : MEDIUM  
Department : Not Assigned  
Status : Registered  
-----
```

3. Assign department:

```
Enter your choice: 3  
  
Enter Complaint ID: 103  
  
Department Assigned Successfully!  
Department: Sanitation Department
```


Enter your choice: 3

Enter Complaint ID: 102

Department Assigned Successfully!
Department: Water Supply Department

Enter your choice: 3

Enter Complaint ID: 101

Department Assigned Successfully!
Department: Road Maintenance Department

Enter your choice: 3

Enter Complaint ID: 105

Department Assigned Successfully!
Department: Electrical Department

Enter your choice: 3

Enter Complaint ID: 104

Department Assigned Successfully!
Department: Drainage Department

4. Update Complaint Status:

```
Enter your choice: 4

Enter Complaint ID: 102

Select New Status:
1. Registered
2. Assigned
3. In Progress
4. Resolved
5. Closed
Enter choice: 4

Status Updated Successfully!
```

5. Track Resolution:

```
Enter your choice: 5

Enter Complaint ID: 101

===== RESOLUTION TRACKING =====
-----
Complaint ID : 101
Citizen Name : ravi kumar
Type         : Road Damage
Description  : large pot hole near college
Location     : chennai main road
Priority     : HIGH
Department  : Road Maintenance Department
Status       : In Progress
-----

Complaint is still under processing.
```

6. Compare Two Complaints:

```
Enter your choice: 6

Enter First Complaint ID: 101
Enter Second Complaint ID: 102

Complaints are different.
Complaint 102 has higher priority.
```

7. Generate Service Performance Report:

```
Enter your choice: 7

=====
      SERVICE PERFORMANCE REPORT
=====
Total Complaints      : 5
Resolved Complaints   : 2
Pending Complaints    : 3

Road Complaints       : 1
Water Complaints      : 1
Waste Complaints      : 1
Drainage Complaints   : 1
Streetlight Complaints : 1

Resolution Rate       : 40%
=====
```

8. Display Statistics:

```
Enter your choice: 8

===== SYSTEM STATISTICS =====
Total Complaints Registered: 5
Current Complaints Stored: 5
```

Result:

The **Urban Complaint Management System** was successfully implemented in C++. The system registered 5 complaints, assigned departments, updated statuses, compared priorities, tracked resolutions, and generated a performance report with a **40% resolution rate**. The required OOP concepts were successfully demonstrated.